G9 - Documentação da Fase 1

Instituição: Pontifícia Universidade Católica de Minas Gerais
Disciplina: Algoritmos e Estruturas de Dados III
Professor: Walisson Ferreira de Carvalho
Integrantes: Fernando Mucci, Maria Eduarda P. Brito, Luan Oliveira, Luiz Felipe Volpe, Luisa Campanha
Demonstração: https://youtu.be/1CYf26WfM6E
Repositório: https://github.com/projetos-puc-minas/TP-AEDS-III-G9/tree/main

---

Definição do Problema

Contexto e Necessidade Real

Problema Identificado: Através de entrevistas com leitores assíduos, identificamos que muitos enfrentam dificuldades para organizar suas bibliotecas pessoais. Os principais desafios relatados incluem:

- Perda de controle sobre quais livros já foram lidos ou estão na lista de desejos.
- Dificuldade em lembrar detalhes de livros (editora, ano, preço pago).
- Impossibilidade de buscar livros por temas específicos (tags).
- Falta de ferramentas para gerenciar o acervo pessoal.
- Necessidade de relacionar livros com seus autores e editoras.

Solução Proposta: Desenvolver um sistema desktop de gerenciamento de biblioteca pessoal que permita catalogar livros, autores e editoras, com funcionalidades avançadas de busca, organização e segurança de dados.

Usuários Entrevistados:

- **Felipe, 19 anos, estudante de Ciências da Computação - Belo Horizonte/MG**

  "Eu organizo meus livros em pastas no computador com arquivos de texto, cada um seguindo um modelo que eu criei com as informações básicas do livro. Funciona, mas é bem trabalhoso de manter atualizado. Já tentei usar o Notion mas achei confuso — as informações ficam espalhadas e filtrar é difícil. O que sinto mais falta é de buscar por tema ou autor sem precisar abrir arquivo por arquivo. Sistema de login seria útil também, tanto pra acessar de qualquer computador quanto por privacidade."

Modelagem de Tabelas e Dados

Tabela: *livros*

Campo	Tipo Java	Descrição
lapide	`boolean`	`true` = ativo, `false` = excluído
tam_registro	`int`	Total de bytes do registro
id	`int`	Identificador único (PK)
id_editora	`int`	FK para Editoras
titulo	`String`	Título do livro
isbn	`char[13]`	Código ISBN
ano_publicacao	`int`	Ano de publicação
preco	`double`	Preço de compra
sinopse	`String`	Sinopse do livro

Tabela: *autores*

Campo	Tipo Java	Descrição
lapide	`boolean`	`true` = ativo, `false` = excluído

Campo	Tipo Java	Descrição
tam_registro	`int`	Total de bytes do registro
id	`int`	Identificador único (PK)
nome	`String`	Nome completo do autor
data_nascimento	`long`	Timestamp Unix
biografia	`String`	Biografia resumida

Tabela: *editoras*

Campo	Tipo Java	Descrição
lapide	`boolean`	`true` = ativo, `false` = excluído
tam_registro	`int`	Total de bytes do registro
id	`int`	Identificador único (PK)
nome	`String`	Nome da editora
cidade	`String`	Cidade sede
ano_fundacao	`int`	Ano de fundação

Tabela: *livros_autores*

Campo	Tipo Java	Descrição
lapide	`boolean`	`true` = ativo, `false` = excluído
tam_registro	`int`	Total de bytes do registro
id	`int`	Identificador único (PK)
id_livro	`int`	FK para Livros
id_autor	`int`	FK para Autores

Tabela: *tags*

Campo	Tipo Java	Descrição
lapide	`boolean`	`true` = ativo, `false` = excluído
tam_registro	`int`	Total de bytes do registro
id	`int`	Identificador único (PK)
nome	`String`	Texto do rótulo temático

Tabela: *tags_livros*

Campo	Tipo Java	Descrição
lapide	`boolean`	`true` = ativo, `false` = excluído
tam_registro	`int`	Total de bytes do registro
id	`int`	Identificador único (PK)
id_livro	`int`	FK para Livros
id_tag	`int`	FK para Tags

Tabela: alugados

Campo	Tipo Java	Descrição
lapide	`boolean`	`true` = ativo, `false` = excluído
tam_registro	`int`	Total de bytes do registro
id	`int`	Identificador único (PK)
id_usuario	`int`	FK para Usuarios
id_livro	`int`	FK para Livros

Tabela: *usuarios*

Campo	Tipo Java	Descrição
lapide	`boolean`	`true` = ativo, `false` = excluído
tam_registro	`int`	Total de bytes do registro
id	`int`	Identificador único (PK)
nome	`String`	Nome do usuário
email	`String`	Email (usado no login)
senha_xor	`String`	Senha criptografada via XOR

Relacionamentos

1:N — Editora publica múltiplos Livros

- Uma editora pode publicar vários livros, mas cada livro pertence a apenas uma editora.
- Implementado através da chave estrangeira `id_editora` na tabela `Livros`.

N:N — Livros possuem múltiplos Autores e Autores escrevem múltiplos Livros

- Um livro pode ter vários autores (co-autoria) e um autor pode escrever vários livros.
- Implementado através da tabela intermediária `Livros_Autores`, com as chaves estrangeiras `id_livro` e `id_autor`.
- A relação entre `Livros` e `Livros_Autores` é **1:N** (possui), e entre `Autores` e `Livros_Autores` também é **1:N** (escreve), compondo juntas a cardinalidade N:N.

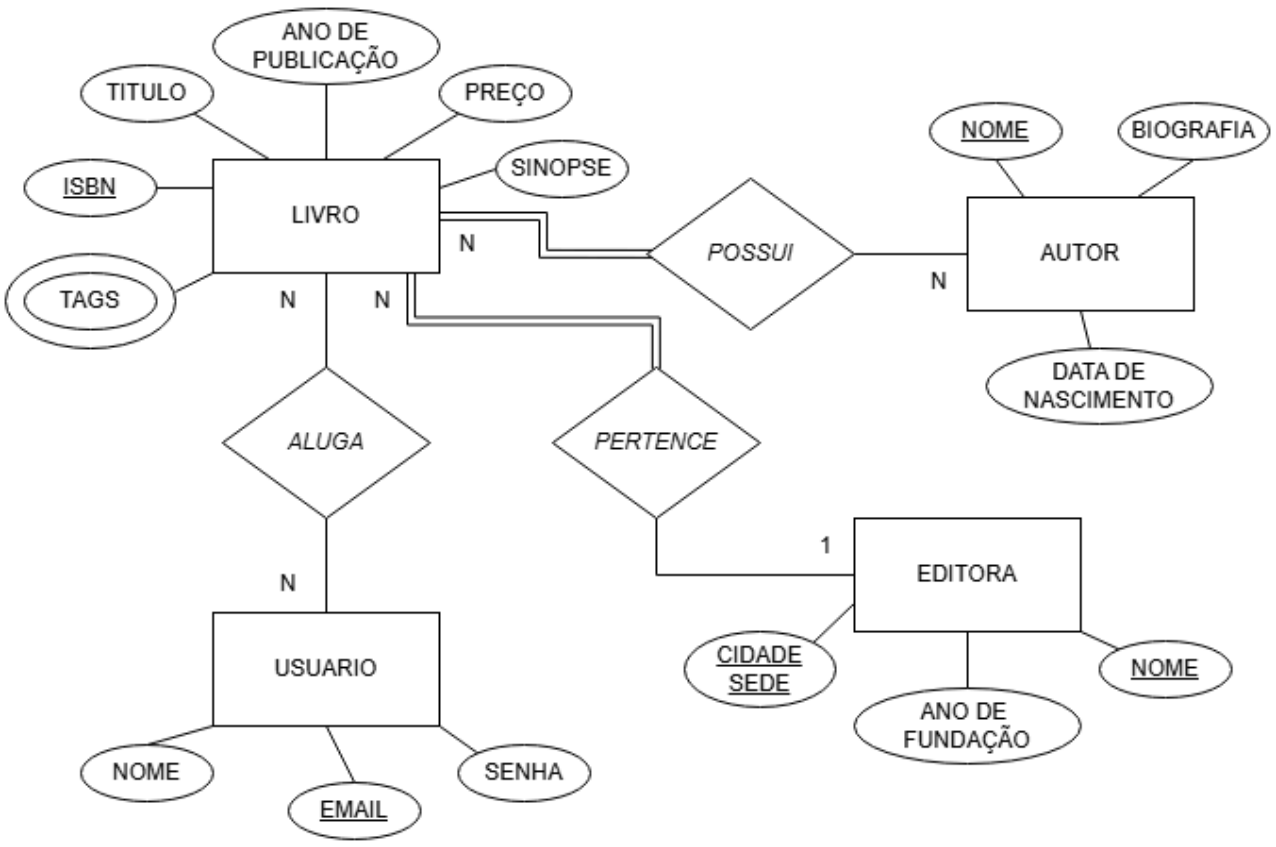
N:N — Livros possuem múltiplas Tags e Tags pertencem a múltiplos Livros

- Um livro pode ter várias tags temáticas, e uma mesma tag pode estar associada a múltiplos livros. A tag é uma **entidade independente**, com identidade e ciclo de vida próprios.
- Implementado através da tabela `Tags_Livros`, que atua como **tabela associativa** entre `Livros` e `Tags`, onde cada registro representa um vínculo entre um livro via `id_livro` e uma tag via `id_tag`.
- A cardinalidade é N:N entre `Livros` e `Tags`: um livro possui muitas tags, e uma tag pode pertencer a muitos livros.

N:N — Usuários gerenciam (alugam) múltiplos Livros

- Um usuário pode ter múltiplos livros associados, e um livro pode estar associado a múltiplos usuários.
- Implementado através da tabela intermediária `Usuarios_Livros`, com as chaves estrangeiras `id_usuario` e `id_livro`.
- A relação entre `Usuarios` e `Usuarios_Livros` é **1:N** (gerencia), e entre `Livros` e `Usuarios_Livros` também é **1:N** (é gerenciado por), compondo juntas a cardinalidade N:N.

Diagrama Entidade Relacionamento



Esquema dos Arquivos Binários (.dat)

Todos os arquivos seguem a mesma estrutura geral: **Cabeçalho** → **Registros**.

Cada registro possui uma **lápide** (exclusão lógica) como primeiro byte.

Estrutura Geral

Posição no Arquivo	Conteúdo	Estrutura dos Dados	Observação
0	Cabeçalho	Metadados do arquivo	Início do arquivo.
—	Registro 0	[lápide;campos...]	Primeiro registro de dados.
—	Registro 1	[lápide;campos...]	Segundo registro de dados.
...	...	...	Demais registros.

---

1. Cabeçalho *(comum a todos os arquivos)*

Offset	Tamanho	Tipo	Campo	Descrição
0	4 bytes	int	`ultimo_id`	Último ID gerado.
4	4 bytes	int	`total_registros`	Qtd. de registros ativos (não deletados).
8	8 bytes	long	`ponteiro_lista_excluidos`	Endereço do primeiro espaço livre na lista encadeada (-1 = vazia).

---

2. livros.bin

Offset	Tamanho	Tipo	Campo
0	1 byte	boolean	`lapide`
1	4 bytes	int	`tam_registro`
5	4 bytes	int	`id`
9	4 bytes	int	`id_editora`
13	tamString + 1 byte (`;`)	string	`titulo`
—	13 bytes	char[13]	`isbn`
—	4 bytes	int	`ano_publicacao`

Offset	Tamanho	Tipo	Campo
—	8 bytes	double	`preco`
—	tamString + 1 byte (`;`)	string	`sinopse`

> Uma editora é uma entidade independente que pode ser responsável pela publicação de diversos títulos ao longo do tempo, mas cada livro possui exatamente uma editora responsável pela sua publicação. Por isso, a chave `id_editora` é armazenada diretamente no registro do livro, refletindo essa dependência direcional.

---

3. autores.bin

Offset	Tamanho	Tipo	Campo
0	1 byte	boolean	`lapide`
1	4 bytes	int	`tam_registro`
5	4 bytes	int	`id`
9	tamString + 1 byte (`;`)	string	`nome`
—	8 bytes	long	`data_nascimento`
—	tamString + 1 byte (`;`)	string	`biografia`

> **Explicação do timestamp:** data convertida em timestamp Unix — quantidade de segundos passados desde 1970 — para facilitar a busca e comparação de datas.

---

4. editoras.bin

Offset	Tamanho	Tipo	Campo
0	1 byte	boolean	`lapide`
1	4 bytes	int	`tam_registro`
5	4 bytes	int	`id`
9	tamString + 1 byte (`;`)	string	`nome`
—	tamString + 1 byte (`;`)	string	`cidade`
—	4 bytes	int	`ano_fundacao`

---

5. livros_autores.bin

Offset	Tamanho	Tipo	Campo
0	1 byte	boolean	`lapide`
1	4 bytes	int	`tam_registro`
5	4 bytes	int	`id`
9	4 bytes	int	`id_livro`
13	4 bytes	int	`id_autor`

> Um único livro pode ter sido escrito por dois ou mais autores, e um mesmo autor pode ter contribuído para vários títulos diferentes. Nenhum dos dois lados da relação pode ser subordinado ao outro, o que exige a tabela intermediária `livros_autores` para registrar cada par (livro, autor) de forma independente.

---

6. tags.bin

Offset	Tamanho	Tipo	Campo
0	1 byte	boolean	`lapide`
1	4 bytes	int	`tam_registro`
5	4 bytes	int	`id`
9	tamString + 1 byte (`;`)	string	`nome`

> Uma tag é uma entidade independente que representa um rótulo temático reutilizável. Por ter identidade própria, pode ser associada a múltiplos livros sem duplicação do valor textual.

---

7. tags_livros.bin

Offset	Tamanho	Tipo	Campo
0	1 byte	boolean	`lapide`
1	4 bytes	int	`tam_registro`
5	4 bytes	int	`id`
9	4 bytes	int	`id_livro`
13	4 bytes	int	`id_tag`

> Um livro pode receber vários rótulos temáticos, e uma mesma tag pode estar associada a múltiplos livros. Como nenhum dos lados pode ser subordinado ao outro, a tabela intermediária `tags_livros` registra cada par (livro, tag) como um vínculo independente via `id_livro` e `id_tag`, caracterizando uma relação N:N.

---

8. alugados.bin

Offset	Tamanho	Tipo	Campo
0	1 byte	boolean	`lapide`
1	4 bytes	int	`tam_registro`
5	4 bytes	int	`id`
9	4 bytes	int	`id_usuario`
13	4 bytes	int	`id_livro`

> Um usuário pode querer alugar vários livros simultaneamente, e um mesmo livro pode estar sendo alugado por múltiplos usuários da plataforma. Como nenhum dos lados determina o outro, a tabela intermediária `alugados` registra cada associação (usuário, livro) como um vínculo independente, sem impor restrições de exclusividade a nenhuma das partes.

---

9. usuarios.bin

Offset	Tamanho	Tipo	Campo
0	1 byte	boolean	`lapide`
1	4 bytes	int	`tam_registro`
5	4 bytes	int	`id`
9	tamString + 1 byte (`;`)	string	`nome`
—	tamString + 1 byte (`;`)	string	`email`
—	tamString + 1 byte (`;`)	string	`senha_xor`